



Course Name: Computer Graphics Fundamentals

Project Title: Assignment 2. Part A: Interaction, Windows, Menus and Animation. Part B: 3D Transformations and Geometry

Team Members: Altynay Yertay, SE-2416

Instructor: Almas Zhanabilov

Date of Submission: 27 September 2026

Table of Contents

1. Introduction.....	3
2. Literature Review.....	4
3. Project Description.....	6
3.1 Objective.....	6
3.2 Problem Statement	6
3.3 Scope	6
4. Technical Implementation	8
4.1 Tools and Technologies Used	8
4.2 Development Process	8
4.3 Algorithms and Techniques	9
5. Graphics and Rendering.....	12
5.1 2D/3D Rendering Techniques.....	12
5.2 Lighting and Shading	23
5.3 Texturing and Animation.....	23
6. Testing and Evaluation	24
6.1 Testing Methodology.....	24
6.2 Performance Evaluation.....	24
6.3 Issues and Debugging.....	25
7. Conclusion and Future Work	26
8. References.....	28
9. Appendix.....	30
9.1 Contents of the submission	30
9.2 How to build and run.....	30
9.3 Controls.....	30
9.4 The vertex shader of Part B	31
9.5 Repository	31

1. Introduction

This assignment continues the first one. Assignment 1 drew still 2D shapes with the programmable OpenGL pipeline. Assignment 2 adds the three things that make a graphics program interactive: several windows, input from the user through menus, the keyboard and the mouse, and change over time.

The work has two parts. **Part A** modifies the “black & white” square picture of Assignment 1. It opens a main window with rotating squares, a subwindow with an ellipse, and a second window called “window 2” with a breathing circle and a rotating triangle. The user controls them with pop-up menus, a submenu, the keyboard and mouse clicks. **Part B** is a separate program that shows the colour cube from the lecture in 3D. The user chooses scaling, rotation or translation in a menu and changes the cube with the keyboard. The program builds the model matrix $M = T * R * S$ on the CPU and applies it to every vertex in the vertex shader.

Both parts use the same structure as Assignment 1: GLUT for windows and input, the InitShader module, GLSL shaders loaded from files, and vertex data stored in VAOs and VBOs.

2. Literature Review

The review is based on 17 journal articles. For every article the authors, year, journal, volume, pages and DOI were checked in the Crossref database, and the journals were checked against the SCImago Journal Rank (SJR). All of them are ranked Q1 or Q2, as summarised in Table 1.

Table 1. Journals of the reviewed articles and their quartiles

Journal	Quartile	Articles
ACM Transactions on Graphics	Q1	Alexa (2002); Blythe (2006); Kovar et al. (2002); Sumner and Popović (2004); Kavan et al. (2008); Lloyd et al. (2008); Jacobson et al. (2011)
Computer Graphics Forum	Q1	Slater et al. (1992); Owens et al. (2007)
IEEE Transactions on Visualization and Computer Graphics	Q1	Cohen-Or et al. (2003)
IEEE Transactions on Pattern Analysis and Machine Intelligence	Q1	Hartley (1997)
Proceedings of the IEEE	Q1	Owens et al. (2008)
Communications of the ACM	Q1	Fatahalian and Houston (2008)
ACM Computing Surveys	Q1	Sutherland, Sproull and Schumacker (1974)
ACM Transactions on Information Systems	Q1	Myers (1990)
ACM Transactions on Computer-Human Interaction	Q1–Q2	Myers (1995); Myers, Hudson and Pausch (2000)

Event-driven interaction and user interface software. Every GLUT program is event-driven: the application registers callback functions and the main loop calls them when an event arrives. Myers (1990) proposed a model in which input is handled by a small set of reusable “interactor” objects instead of code written again for every widget, which is the idea behind separating the display, keyboard, mouse and menu callbacks in this assignment. Myers (1995) surveyed user interface software tools and described how toolkits hide the event loop behind callbacks, and Myers, Hudson and Pausch (2000) reviewed which of these ideas succeeded, with pop-up menus and callback-based toolkits among the lasting ones. GLUT menus, submenus and per-window callbacks follow this model directly.

The programmable pipeline. Blythe (2006) described the design of the Direct3D 10 pipeline, in which fixed functions are replaced by programmable vertex and pixel stages; OpenGL 3.2 core, used here, follows the same design. Owens et al. (2007) and Owens et al. (2008) surveyed how the stages of the graphics pipeline are mapped to highly parallel graphics hardware, and Fatahalian and Houston (2008) explained why a GPU processes thousands of vertices and fragments at the same time. These works explain why this project uploads geometry once and changes only a few uniform values per frame: the vertex shader applies the same transformation to all vertices in parallel.

Geometric transformations and homogeneous coordinates. Part B represents points in homogeneous coordinates $(x, y, z, 1)$, so that translation becomes a matrix product like scaling and rotation. Hartley (1997) showed in computer vision that working with homogeneous coordinates is powerful but needs care with numerical conditioning. Alexa (2002) studied how transformations can be combined and interpolated, and pointed out that matrix products are not commutative; this is exactly why the order $T * R * S$ matters in Part B. Kavan et al. (2008) compared ways of blending rotations and showed the artefacts that appear when rotation matrices are combined naively. Jacobson et al. (2011) and Sumner and Popović (2004) used per-vertex combinations of affine transformations to deform shapes in real time, which is the general form of the per-vertex matrix product used in the vertex shader of Part B.

Animation. Kovar, Gleicher and Pighin (2002) built animations by connecting short motion clips in a graph and playing them over time. Like every animation system, it updates the state of the scene from time, not from the number of frames drawn. Part A follows the same rule on a small scale: the idle function reads the real elapsed time and computes the angles and the scale from it.

Hidden surface removal and the depth buffer. Sutherland, Sproull and Schumacker (1974) classified ten hidden-surface algorithms and described the depth buffer (z-buffer) as the simplest one, whose cost does not depend on sorting. Slater et al. (1992) compared the z-buffer with the scan-line algorithm statistically and showed when each is faster. Cohen-Or et al. (2003) surveyed visibility methods that reduce the work before the z-buffer, and Lloyd et al. (2008) studied how depth values are distributed and how their precision affects the result. Part B uses the plain z-buffer: `GL_DEPTH_TEST` keeps the nearest fragment of the cube.

3. Project Description

3.1 Objective

The objective is to turn the still pictures of Assignment 1 into interactive and animated programs, and to apply 3D geometric transformations to a cube with a matrix computed in the application.

The concrete goals are the following:

- enable double buffering;
- show the “black & white” square of Assignment 1 in a main window, an ellipse in a subwindow with its own background, and a circle and a triangle in a separate window titled “window 2”;
- add a menu in the subwindow for its background colour, and a main-window menu with Stop Animation, Start Animation and a Square Colors submenu;
- change the colour of the circle and the triangle with the keys r, g, b, y, o, p and w in window 2;
- animate the shapes with one idle function: squares counter-clockwise, triangle clockwise, circle breathing;
- extra credit: add a breathing circle of a random colour at every left click;
- in Part B, show the colour cube with depth testing, let the user choose SCALE, ROTATE or TRANSLATE in a menu, change X, Y and Z with six keys, change the step (delta) with two keys, reset with one key, and apply $M = T * R * S$ in the vertex shader.

3.2 Problem Statement

The first problem is that GLUT is event-driven. The program does not decide when to draw or read input; it only registers callbacks, and GLUT calls them. Every window and subwindow has its own callbacks and, on macOS, its own OpenGL context. A shader program, a VAO or a VBO created in one context cannot be used in another, so every window needs its own copies.

The second problem is animation without rebuilding geometry. Rotating or breathing shapes could be drawn by recomputing all vertices every frame, but that wastes work and is not how the pipeline is meant to be used. The geometry must stay in the VBO, and only a few uniform values (an angle, a scale and an offset) may change per frame. The speed must also be the same on fast and slow computers, so the animation must depend on real time.

The third problem is the order of 3D transformations. Matrix multiplication is not commutative: scaling after a rotation stretches the cube along the rotated axes, and rotating after a translation swings the cube around the origin. The assignment fixes the order: scale first, then rotate, then translate. It also forbids a camera or a projection, so the cube is drawn in the default orthographic view, where translation along Z must not change the size of the cube.

3.3 Scope

The following is included:

- three GLUT windows (one of them a subwindow) with separate contexts, shader programs, VAOs and VBOs;
- pop-up menus attached to the right mouse button, one of them with a submenu;
- keyboard and mouse callbacks, including conversion of mouse coordinates to normalized device coordinates;
- animation with one idle function based on `GLUT_ELAPSED_TIME`, with rotation and scaling done in the vertex shader through uniforms;
- a 3D cube with per-vertex colours, depth testing and a 4×4 model matrix built on the CPU.

The following is excluded, as required by the assignment or because it belongs to later topics:

- cameras, viewing matrices and perspective projection;
- lighting, normals and textures;
- animation in Part B (the cube changes only when a key is pressed).

4. Technical Implementation

4.1 Tools and Technologies Used

Table 2. Tools and technologies

Component	Value and purpose
Programming language	C++, standard C++11 (same as Assignment 1)
Graphics API	OpenGL 3.2 core profile, requested with <code>GLUT_3_2_CORE_PROFILE</code>
Shading language	GLSL 1.50 (#version 150)
Windowing, menus and input	GLUT, provided by macOS as a system framework
Shader loading	InitShader.cpp from Assignment 1, unchanged
Compiler	Apple clang 21.0.0
Build system	GNU make, one Makefile with targets <code>partA</code> and <code>partB</code>
Operating system	macOS 26.5.1 on Apple Silicon (Apple M4)
Version control	Git, with the repository hosted on GitHub

GLUT was chosen because the assignment needs features that it provides directly: `glutCreateSubWindow`, `glutCreateMenu`, `glutAddSubMenu` and `glutIdleFunc`. On macOS GLUT and OpenGL are system frameworks, so nothing had to be installed. The Makefile also builds on Linux with `freeglut` and `GLEW`, like the Makefile of Assignment 1.

4.2 Development Process

The work was done step by step, and every step ended with a program that built and ran.

1. **Starting point.** The code of Assignment 1 was studied: the `InitShader` module, the shaders, the Makefile and `picture.cpp`, where the “black & white” square is six nested squares drawn with `GL_TRIANGLE_FAN` in alternating white and black. `InitShader.cpp`, `InitShader.h` and `fshader.glsl` were copied without changes.
2. **One scene per window.** Because every GLUT window has its own context, the geometry and the GL objects of a window were grouped in a `Scene` structure, and `initScene()` creates the VAO, the VBO and the shader program for the window that is current at that moment.
3. **Windows.** The main window with the squares, the subwindow with the ellipse and window 2 with the circle and the triangle were created, each with its own display function.
4. **Animation.** A vertex shader with three uniforms (`theta`, `scale`, `offset`) was written, and one idle function was added that computes the angles and the scale from the elapsed time.
5. **Menus and keyboard.** The main-window menu with its submenu, the subwindow menu and the keyboard callback of window 2 were added.
6. **Extra credit.** A left click in the main window adds a breathing circle of a random colour at the click position.

7. **Part B.** The colour cube of the lecture, a small mat4 type with the matrix functions, the menu, the keyboard callback and the instructions printed at start-up were written.
8. **Testing.** Every requirement was tested, and screenshots were taken (section 6.1).

The project was developed individually, so no division of work between team members was required.

4.3 Algorithms and Techniques

One scene per window. Each window has its own vertex and colour arrays, its own buffer and its own uniform locations. The same initScene() function is called once per window, right after the window is created, so the objects are created in the correct context.

```
struct Scene {
    GLfloat vertices[MAX_VERTICES][4];
    GLfloat colors[MAX_VERTICES][4];
    int numVertices;
    GLuint buffer;
    GLint thetaLoc, scaleLoc, offsetLoc;
};

Scene mainScene, subScene, scene2;
```

Rotation and breathing in the vertex shader. The geometry of every shape is stored once, centred at the origin. The vertex shader scales it, rotates it by theta and moves it to offset. A positive angle rotates counter-clockwise and a negative angle rotates clockwise.

```
float c = cos(theta);
float s = sin(theta);
vec2 p = scale * vPosition.xy;
p = vec2(c * p.x - s * p.y, s * p.x + c * p.y);
gl_Position = vec4(p + offset, vPosition.z, 1.0);
```

One idle function driven by time. GLUT has only one idle callback for the whole application. It adds the real time since the previous call to animTime, computes the three animated values from it and asks both animated windows to redraw. Because the values depend on time and not on the number of frames, the speed is the same at any frame rate.

```
void idle()
{
    int now = glutGet(GLUT_ELAPSED_TIME);
    animTime += (now - lastTime) / 1000.0f;
    lastTime = now;

    squareTheta = 1.0f * animTime;
    triangleTheta = -1.5f * animTime;
    breathScale = 1.0f + 0.25f * sin(3.0f * animTime);

    glutPostWindowRedisplay(mainWindow);
    glutPostWindowRedisplay(window2);
}
```

Stop and Start. Stop Animation calls `glutIdleFunc(NULL)`, so the idle function is no longer called and the picture freezes. Start Animation first sets `lastTime` to the current time and then sets the idle function again. Without resetting `lastTime`, the first call after a long pause would add the whole pause to `animTime`, and the shapes would jump.

```
void startAnimation()
{
    lastTime = glutGet(GLUT_ELAPSED_TIME);
    glutIdleFunc(idle);
}
```

Menus and a submenu. The submenu must be created first, because `glutAddSubMenu` needs its identifier. The main menu is then attached to the right mouse button of the main window. The subwindow gets its own menu in the same way, so right-clicking the subwindow opens only the background menu.

```
int colorMenu = glutCreateMenu(squareColorMenu);
glutAddMenuEntry("White", SQUARE_WHITE);
glutAddMenuEntry("Red", SQUARE_RED);
glutAddMenuEntry("Green", SQUARE_GREEN);

glutCreateMenu(mainMenu);
glutAddMenuEntry("Stop Animation", MENU_STOP);
glutAddMenuEntry("Start Animation", MENU_START);
glutAddSubMenu("Square Colors", colorMenu);
glutAttachMenu(GLUT_RIGHT_BUTTON);
```

Square Colors logic. In Assignment 1 the six squares alternate: squares 0, 2 and 4 are white and squares 1, 3 and 5 are black. The chosen colour replaces the **white** squares only, and the black squares stay black, so the picture keeps its pattern of frames. “White” restores the original picture. The colours are written into the colour part of the VBO with `glBufferSubData` only when a menu item is chosen, not every frame.

```
for (int i = 0; i < 6; i += 2)
    setColor(mainScene, squaresStart + i * 4, 4, colors[id][0], colors[id][1], colors[id][2]);
uploadColors(mainScene, squaresStart, 24);
```

Mouse coordinates to NDC. GLUT gives the mouse position in pixels with the origin in the top-left corner and Y growing downwards. Normalized device coordinates go from -1 to 1 with Y growing upwards, so Y must be flipped.

```
clickCenters[numClicks][0] = 2.0f * x / width - 1.0f;
clickCenters[numClicks][1] = 1.0f - 2.0f * y / height;
```

The colour cube. The cube of the lecture has 8 corners with 8 colours (black, red, yellow, green, blue, magenta, white and cyan). `quad()` turns one face into two triangles, and `colorcube()` builds the 6 faces, 36 vertices in total. Every vertex keeps the colour of its corner, so each face shows a gradient from corner to corner.

```

void colorcube()
{
    quad(1, 0, 3, 2);
    quad(2, 3, 7, 6);
    quad(3, 0, 4, 7);
    quad(6, 5, 1, 2);
    quad(4, 5, 6, 7);
    quad(5, 4, 0, 1);
}

```

The model matrix $M = T * R * S$. The matrix is built on the CPU every time the window is redrawn. The rotation is $R = R_z * R_y * R_x$, so a vertex is first rotated about X, then about Y, then about Z. The whole product is applied to a column vector v as $M v = T (R (S v))$: the vertex is scaled first, then rotated, then translated.

```

mat4 S = scaleMatrix(scaleXYZ[0], scaleXYZ[1], scaleXYZ[2]);
mat4 R = multiply(rotateZ(rotateXYZ[2]), multiply(rotateY(rotateXYZ[1]),
rotateX(rotateXYZ[0])));
mat4 T = translateMatrix(translateXYZ[0], translateXYZ[1], translateXYZ[2]);
mat4 M = multiply(T, multiply(R, S));

glUniformMatrix4fv(modelLoc, 1, GL_TRUE, &M.m[0][0]);

```

The matrix is stored row by row in C++, while GLSL expects columns, so it is sent with `transpose = GL_TRUE`. The vertex shader only multiplies: `gl_Position = model * vPosition;`

Keys and deltas. The current transformation is chosen in the menu. Six keys add or subtract its delta on X, Y or Z. + doubles and - halves the delta of the current transformation, and r returns to the initial view: scale 1, rotation -45° about X and 35° about Y, and no translation. Each transformation has its own delta: 0.1 for scale, 10 degrees for rotation and 0.1 for translation.

```

case 'q': v[0] += delta[mode]; break;
case 'a': v[0] -= delta[mode]; break;
case '+':
case '=': delta[mode] *= 2.0f; break;
case '-': delta[mode] *= 0.5f; break;
case 'r': resetTransformations(); break;

```

5. Graphics and Rendering

5.1 2D/3D Rendering Techniques

Both parts use the rendering path of Assignment 1. Positions and colours are stored one after another in one VBO; the attribute pointers describe the layout; the vertex shader receives `vPosition` and `vColor`; the fragment shader writes the interpolated colour. The difference is in the vertex shaders: in Part A they rotate, scale and move the shape with three uniforms, and in Part B they multiply every vertex by the model matrix.

The screenshots of the program windows were taken with the macOS screencapture tool by window. To produce every state reliably, a temporary test copy of each program called the same callback functions that the menus, the keyboard and the mouse call (for example `squareColorMenu(SQUARE_RED)` or `keyboard2('o', 0, 0)`). The menus (Fig.2, Fig.3, Fig.4 and Fig.19) were opened in the normal programs with scripted right clicks and mouse moves (macOS `CGEvent`), and the area of the window with the open menu was captured with `screencapture -R`. Fig.20 shows the real Terminal window after running `./partB`.

Part A

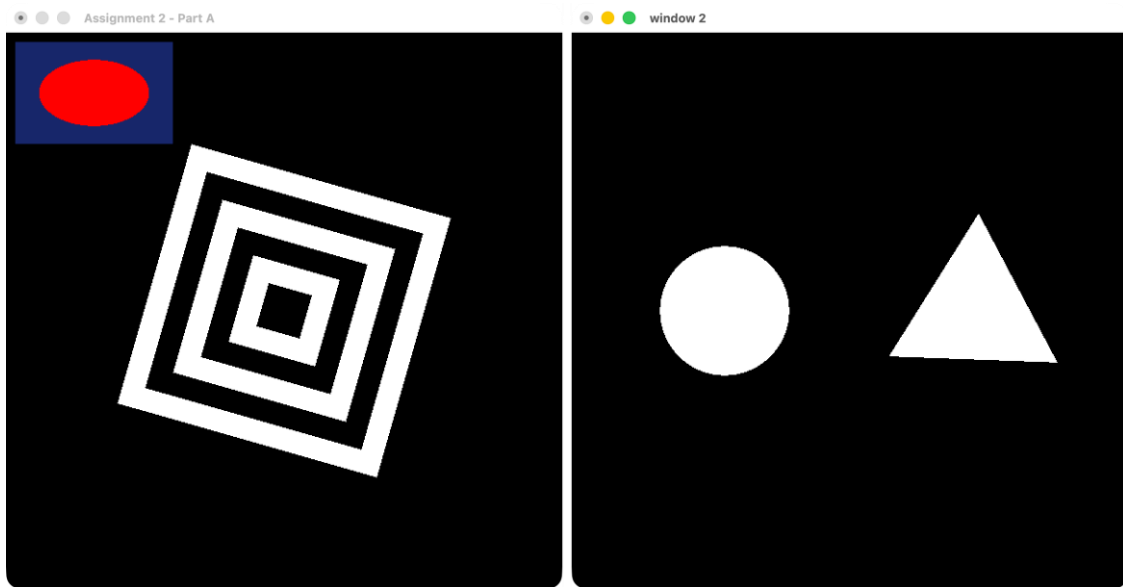


Fig.1 The main window with the rotating squares and the subwindow, and window 2 with the circle and the triangle

Figure 1 illustrates the program right after start-up. The main window shows the six squares of Assignment 1 rotating counter-clockwise, and the subwindow in its top-left corner shows the red ellipse on a dark blue background. Window 2 shows the white breathing circle and the white triangle rotating clockwise.

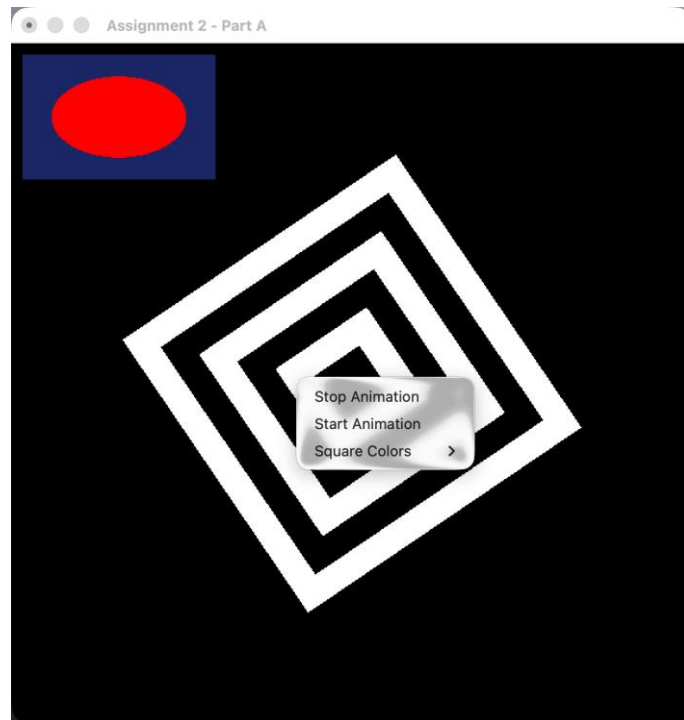


Fig.2 The menu of the main window

Figure 2 illustrates the pop-up menu that opens with the right mouse button in the main window: Stop Animation, Start Animation and the Square Colors submenu.

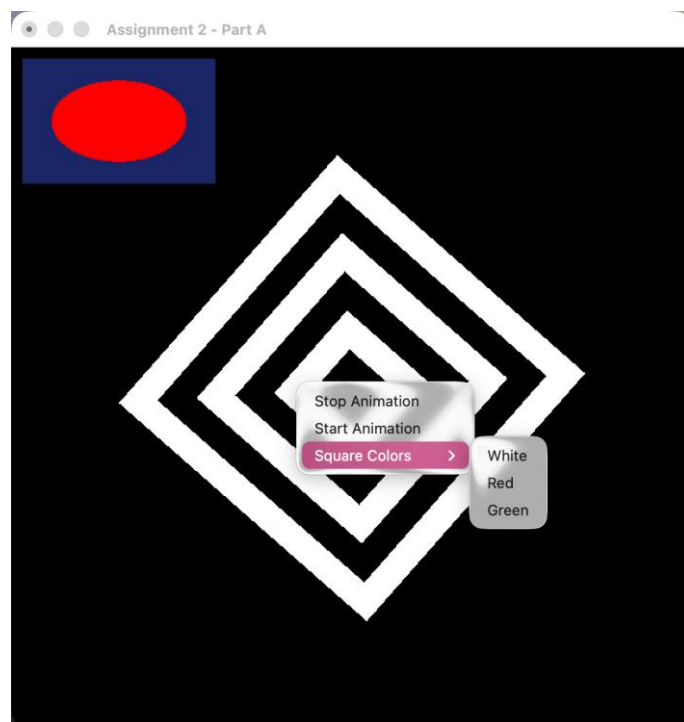


Fig.3 The Square Colors submenu

Figure 3 illustrates the Square Colors submenu with the items White, Red and Green.

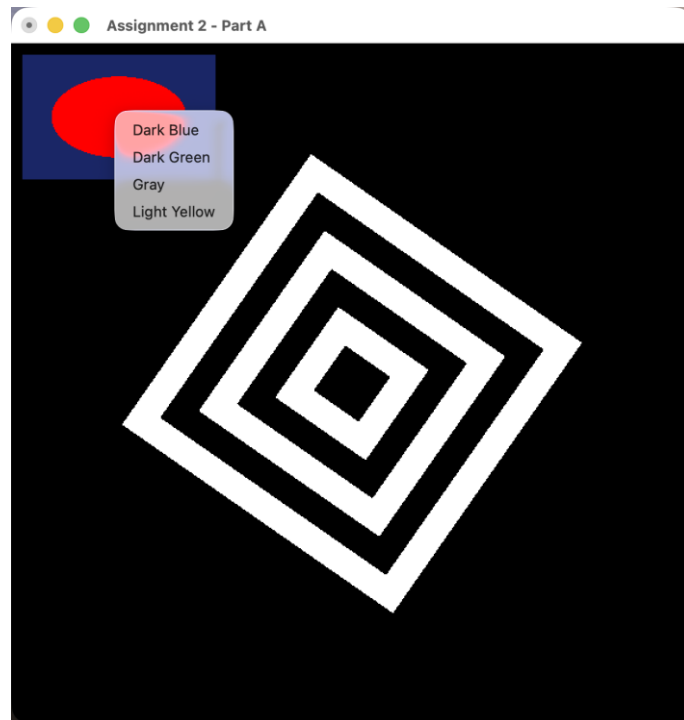


Fig.4 The menu of the subwindow

Figure 4 illustrates the menu of the subwindow. It opens only when the right mouse button is pressed inside the subwindow and offers four background colours.

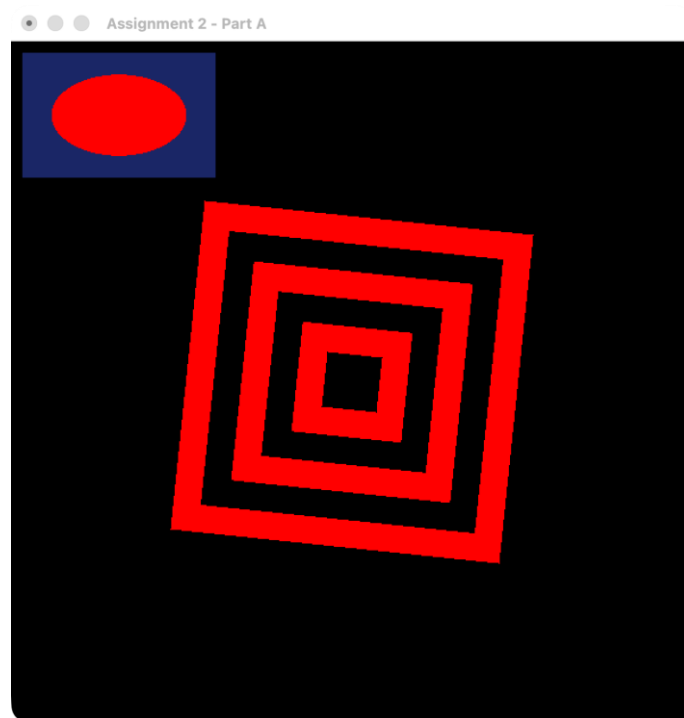


Fig.5 The squares after choosing Square Colors → Red

Figure 5 illustrates the result of choosing Red: the white squares become red, while the black squares stay black, so the frames keep their pattern.

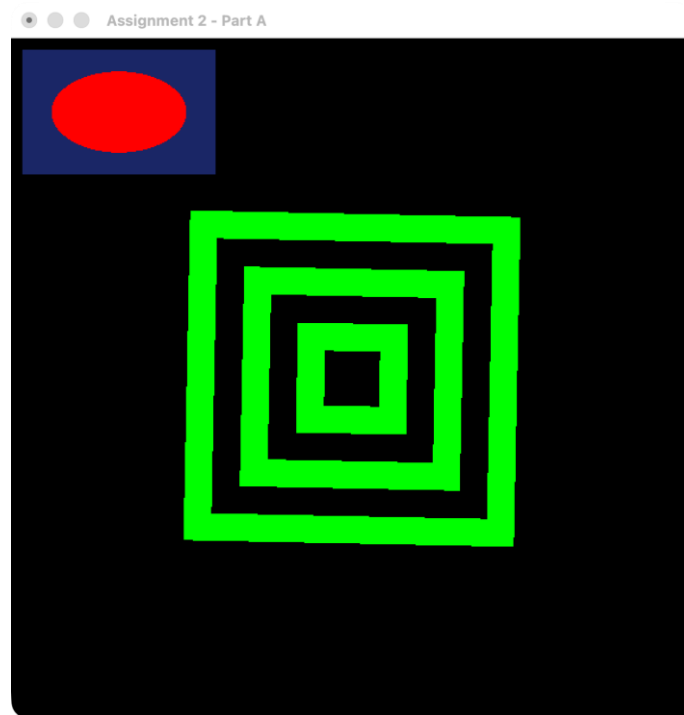


Fig.6 The squares after choosing Square Colors → Green

Figure 6 illustrates the same squares after choosing Green. Choosing White returns the original black and white picture.



Fig.7 Window 2 after pressing the 'o' key

Figure 7 illustrates window 2 after pressing 'o': the circle and the triangle are both orange, because the key changes the colour of the whole scene of window 2.

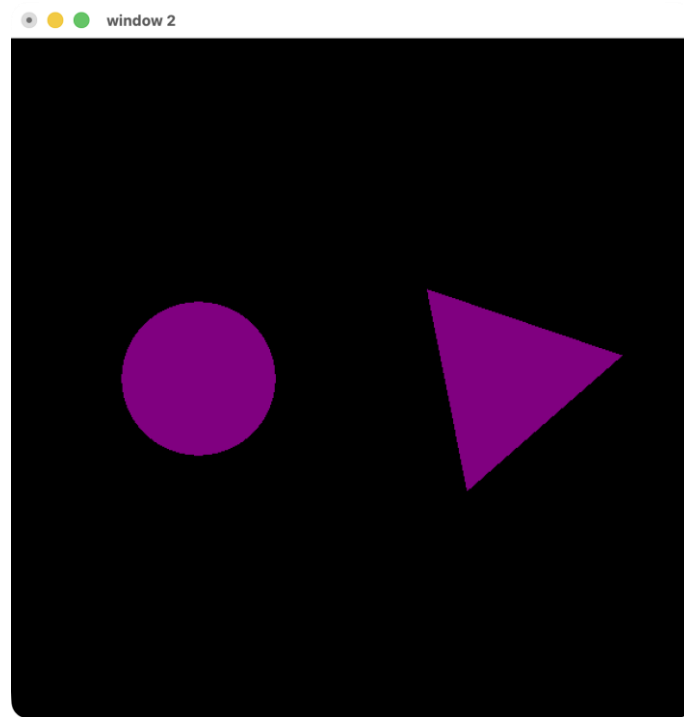


Fig.8 Window 2 after pressing the 'p' key

Figure 8 illustrates window 2 after pressing 'p'. The circle is smaller than in Fig.7 because it is breathing, and the triangle has rotated further clockwise.

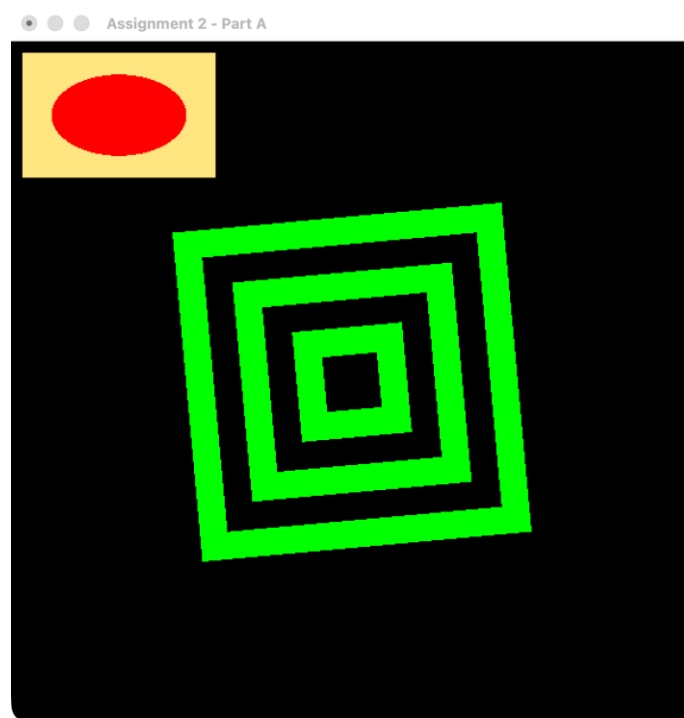


Fig.9 The subwindow after choosing Light Yellow in its menu

Figure 9 illustrates the subwindow with a light yellow background. Only the subwindow changed; the main window keeps its black background.

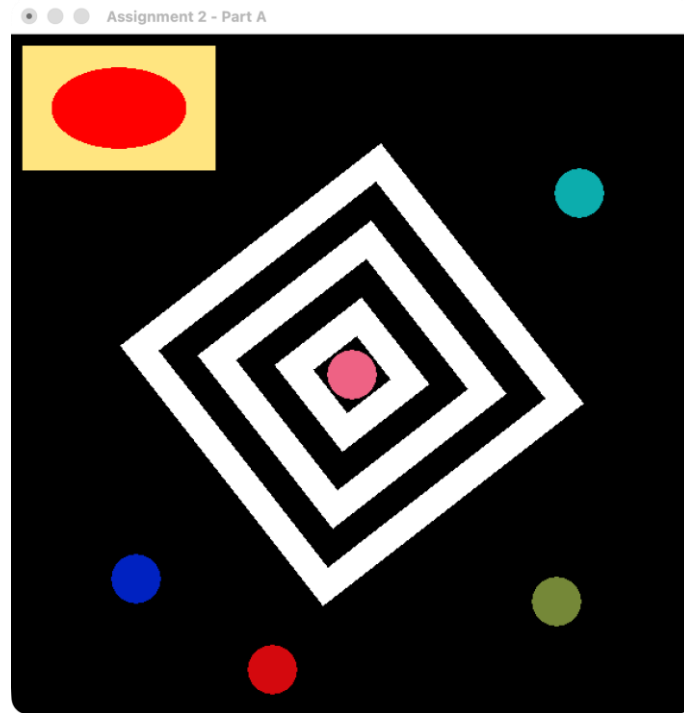


Fig.10 Breathing circles of random colours added by left clicks

Figure 10 illustrates the extra credit: five left clicks added five circles of random colours exactly at the clicked points, including one in the centre on top of the squares. All circles breathe together with the circle in window 2.

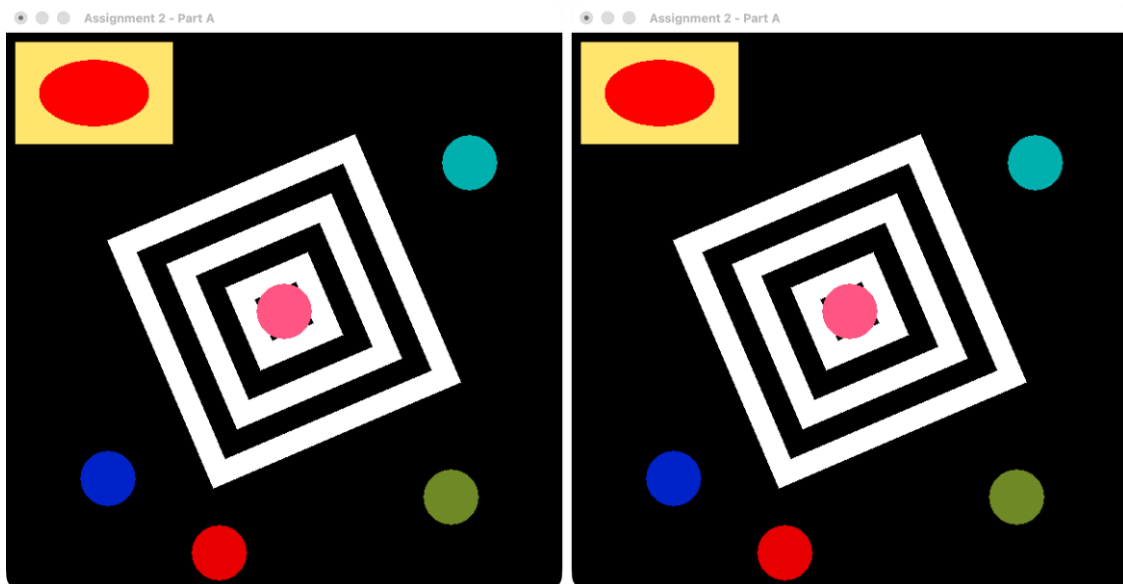


Fig.11 Two frames taken two seconds apart after Stop Animation

Figure 11 illustrates that Stop Animation really stops the idle function: the two frames were captured two seconds apart and are identical byte for byte.

Part B



Fig.12 The cube at start-up: rotated -45° about X and 35° about Y

Figure 12 illustrates the initial view of the cube, the same as in the picture of the assignment. The program starts with a rotation of -45° about X and 35° about Y, which turns the white corner towards the viewer and places it in the centre of the window. Three faces meet at this corner, each with a gradient between the colours of its corners, and the depth test hides the three back faces. Because the rotation about X is applied first ($R = R_z * R_y * R_x$), X needs -45° and Y needs 35° .

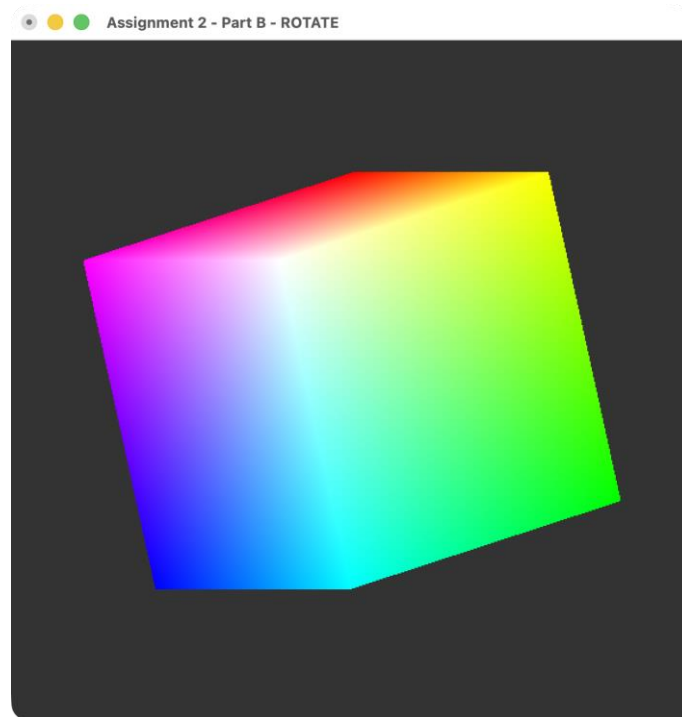


Fig.13 The cube after ROTATE: X changed from -45° to -15° and Y from 35° to 55°

Figure 13 illustrates the cube after pressing q three times and w twice in ROTATE mode. Each press adds the rotation delta of 10° , so the white corner moves away from the centre and the cube turns to show more of its green and cyan faces.

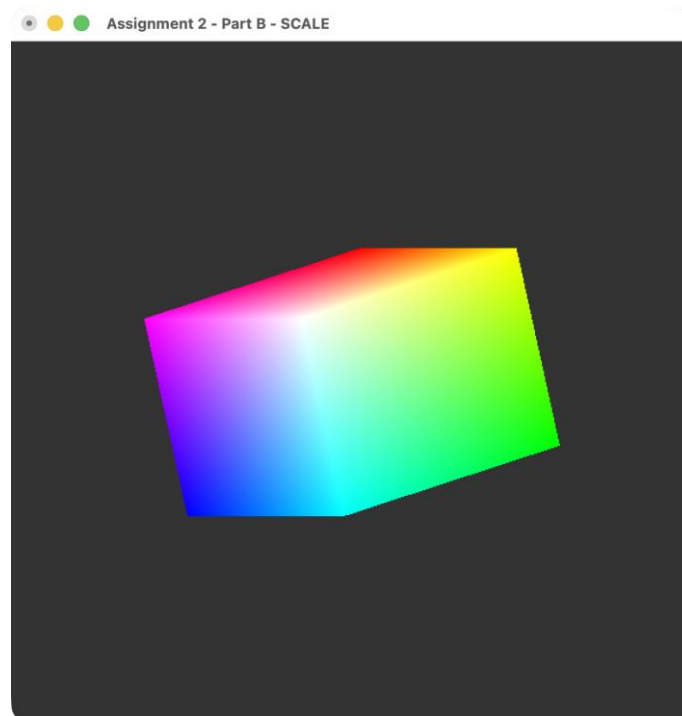


Fig.14 The cube after SCALE: 0.8 on X, 0.6 on Y and 0.8 on Z

Figure 14 illustrates the cube after scaling. Because scaling is applied before the rotation ($M = T * R * S$), the cube shrinks along its own axes and stays a box; it is not sheared.

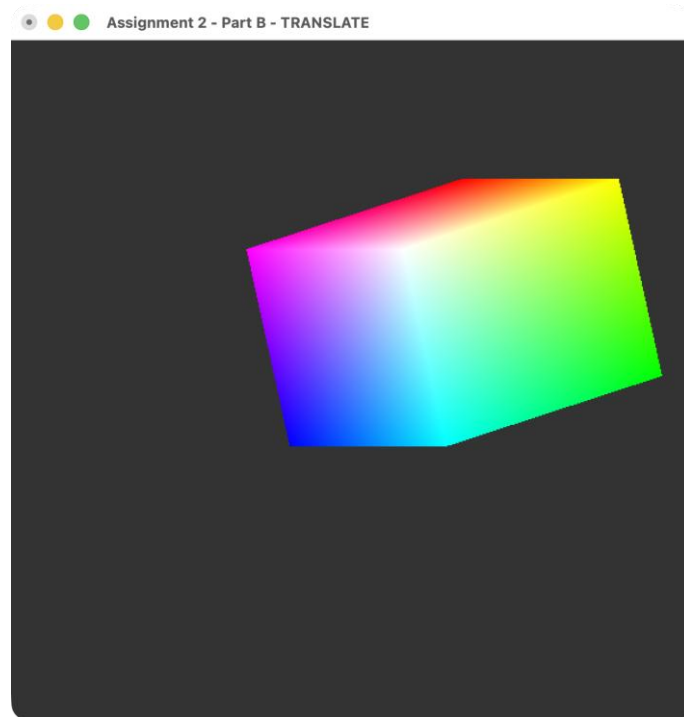


Fig.15 The cube after TRANSLATE: 0.3 on X and 0.2 on Y

Figure 15 illustrates the cube moved to the right and up. Translation is applied last, so the cube keeps its scale and orientation and only moves.

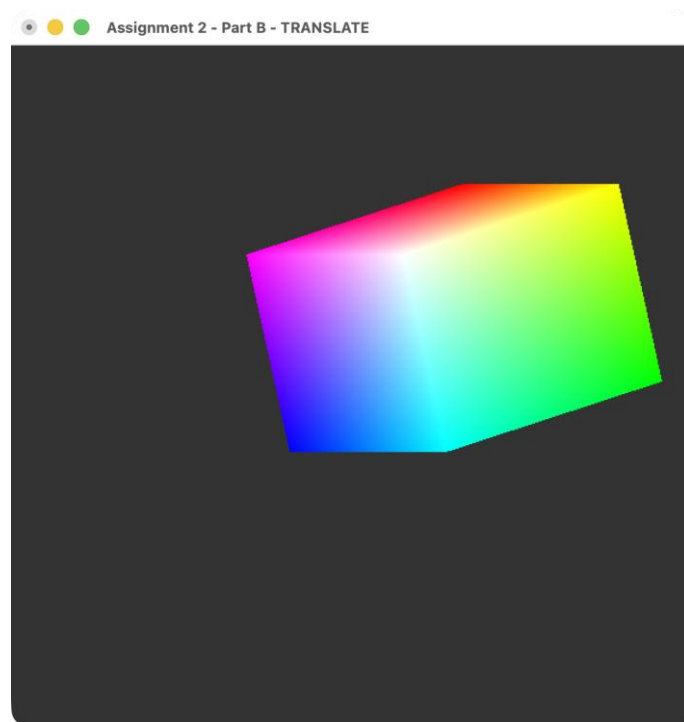


Fig.16 The cube after an extra translation of 0.2 along Z

Figure 16 illustrates that translation along Z does not change the size of the cube in the orthographic view. This image is identical byte for byte to Fig.15.

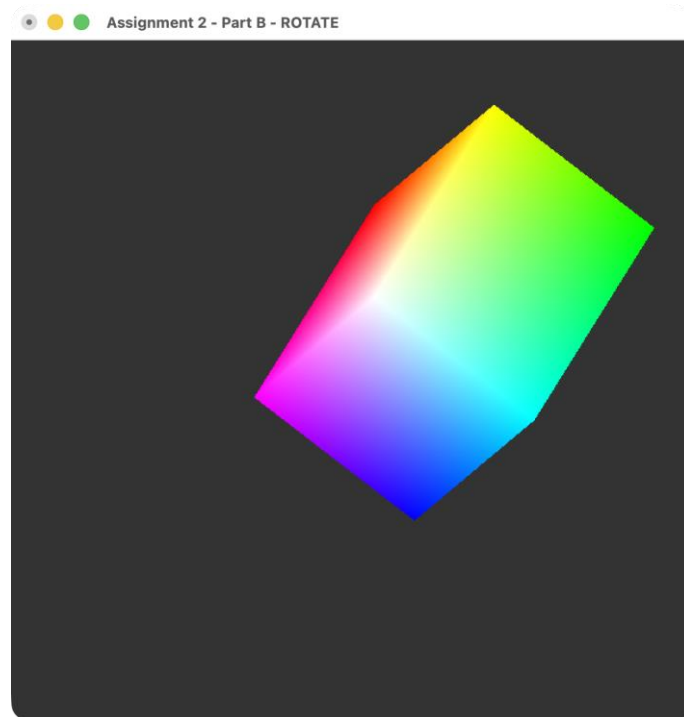


Fig.17 The cube after doubling the rotation delta and rotating about Z

Figure 17 illustrates the delta keys: in ROTATE mode '+' doubled the delta from 10° to 20° , and two presses of 'e' rotated the cube by 40° about Z.



Fig.18 The cube after pressing r: back to the initial view with the white corner towards the viewer

Figure 18 illustrates the reset key: the cube returns to the initial view (scale 1, rotation -45° about X and 35° about Y, no translation) and the deltas return to their default values. The white corner is again in the centre and three faces are visible. The image is identical byte for byte to Fig.12.

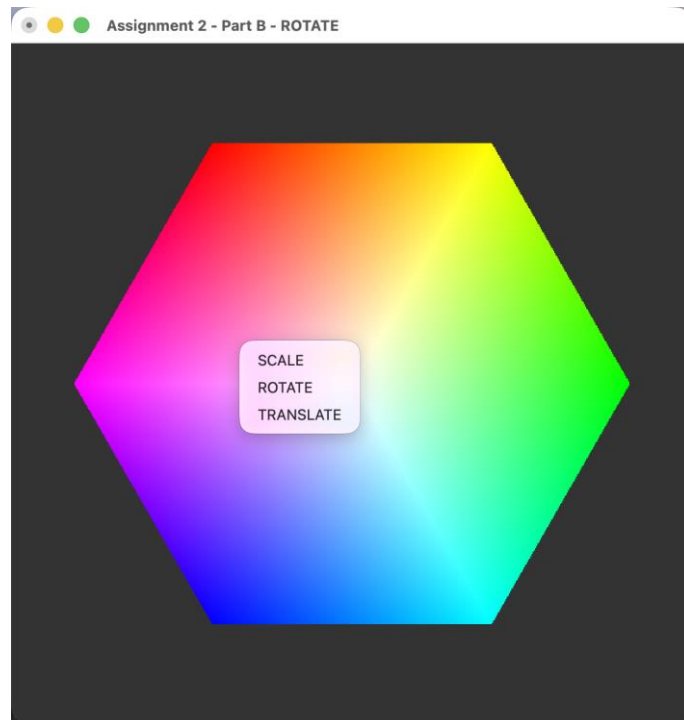


Fig.19 The menu for choosing the transformation

Figure 19 illustrates the menu with SCALE, ROTATE and TRANSLATE. The current transformation is also shown in the window title.

```
altynayyertay — partB — 80x24

Assignment 2 - Part B: 3D cube transformations

Right mouse button: menu to choose the transformation
    SCALE, ROTATE or TRANSLATE (the current one is shown in the title)

Keys for the current transformation:
q / a    increase / decrease X
w / s    increase / decrease Y
e / d    increase / decrease Z
+ / -    double / halve the delta of the current transformation
r        reset to the initial view and the default deltas
Esc      quit

Initial view: rotate X -45, Y 35 degrees (white corner towards the viewer)
Default deltas: scale 0.1, rotate 10 degrees, translate 0.1
The model matrix is  $M = T * R * S$  with  $R = R_z * R_y * R_x$ 
█
```

Fig.20 The instructions printed in the terminal at start-up

Figure 20 illustrates the only output of Part B: the list of keys and menu items printed once when the program starts.

5.2 Lighting and Shading

There is no lighting model in this assignment: no light sources, no normals and no reflection equations. Shading comes only from colour interpolation. In Part B each vertex of the cube has the colour of its corner, and the rasteriser blends the colours across every triangle, so each face shows a smooth gradient from corner to corner. Because the colours depend on the corners and not on light, a face looks the same from any angle.

Hidden surfaces in Part B are removed with the depth buffer. The window is created with `GLUT_DEPTH`, `glEnable(GL_DEPTH_TEST)` is called once, and both the colour and the depth buffer are cleared before every frame:

```
| glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

Part A is flat 2D, so it does not need a depth buffer; the order of the draw calls decides what is on top, as in Assignment 1.

5.3 Texturing and Animation

No textures are used in either part.

Animation is used only in Part A, as the assignment requires. Three values change over time: the angle of the squares (counter-clockwise, 1 radian per second), the angle of the triangle (clockwise, 1.5 radians per second) and the breathing scale $1 + 0.25 \cdot \sin(3t)$, which makes the circles grow and shrink smoothly between 75% and 125% of their size. The subwindow ellipse is not animated. All three values are computed in the idle function from the real elapsed time, and the geometry in the VBOs never changes during the animation.

Part B has no animation. The cube is redrawn only after a key press or a menu choice, through `glutPostRedisplay()`.

6. Testing and Evaluation

6.1 Testing Methodology

Requirement check. Every requirement was tested in the running programs: menus, the submenu, the subwindow menu, all seven colour keys of window 2, Stop and Start, left clicks, the six transformation keys in all three modes, the delta keys and the reset key.

Screenshots as test records. To make the tests repeatable, a temporary test copy of each program called the real callback functions on a timer, and the window was captured by its window identifier with screenshot -l. Two checks were made by comparing image files byte for byte:

- the two frames taken two seconds after Stop Animation are identical, so the idle function really stopped (Fig.11);
- the frames before and after a translation of 0.2 along Z are identical, so Z translation does not change the size of the cube (Fig.15 and Fig.16);
- the frame after pressing r is identical to the frame at start-up (Fig.12 and Fig.18).

Build check. Both programs build with -Wall without any warnings:

```
make clean && make
```

Clean build of the submission. The archive was unpacked into an empty folder, built there with make, and both programs were started from that folder, which repeats what the grader will do.

6.2 Performance Evaluation

The workload of both programs is very small. Table 3 lists the vertices and the draw calls per frame.

Table 3. Rendering workload per frame

Window	Vertices in the VBO	Draw calls per frame
Part A, main window	24 for the squares + 62 per clicked circle	6 + 1 per clicked circle
Part A, subwindow	62 for the ellipse	1
Part A, window 2	62 for the circle + 3 for the triangle	2
Part B	36 for the cube	1

Three properties keep the cost low. The geometry is uploaded once, and the animation changes only three uniform values per draw call. Colour changes rewrite only the affected part of the colour array with `glBufferSubData`, and only when the user chooses a colour. A clicked circle is uploaded once, at the moment of the click. In Part B the 4×4 matrix is computed once per redraw on the CPU (a few dozen multiplications), and the GPU applies it to the 36 vertices.

The idle function runs as often as GLUT allows, but because the animation is based on `GLUT_ELAPSED_TIME`, a faster or slower computer changes only the smoothness, not the speed of the motion.

6.3 Issues and Debugging

Table 4. Issues encountered and their resolution

Issue	Cause and resolution
The instructions of Part B did not appear when the output was redirected to a file.	When standard output is not a terminal, <code>printf</code> buffers it, and the text was lost when the program was closed. <code>fflush(stdout)</code> was added after the instructions.
A screenshot of the screen area around the windows could include other applications if a window moved.	Screenshots are taken by window identifier (<code>screencapture -l</code>), which captures only the program window.
In an early test, the translated cube touched the edge of the window.	The test order was changed to rotate, scale and then translate, which also matches the order of $M = T * R * S$.
The name <code>index</code> for the vertex counter of Part B could clash with the POSIX function <code>index()</code> on Linux.	The variable was renamed to <code>numPoints</code> .

The following points were handled in the design from the start:

Point	How it is handled
Each GLUT window has its own OpenGL context.	<code>initScene()</code> is called right after each window is created, so each window gets its own program, VAO and VBO.
A menu callback changes the buffer of the main window.	<code>squareColorMenu</code> calls <code>glutSetWindow(mainWindow)</code> before <code>glBufferSubData</code> , so the correct context is current.
After Stop and Start the shapes could jump.	<code>startAnimation()</code> resets <code>lastTime</code> before setting the idle function again.
Mouse Y grows downwards.	The NDC formula flips it: $y = 1 - 2 \cdot y / \text{height}$.
A C++ matrix is stored by rows, GLSL expects columns.	The matrix is sent with <code>transpose = GL_TRUE</code> .
Translating far along Z can clip the cube.	Without a projection, only $-1 \leq z \leq 1$ is visible. A cube moved beyond this range is cut by the near or far plane; its size never changes.

7. Conclusion and Future Work

Both parts of the assignment were implemented with GLUT, the InitShader module, GLSL shaders and VAOs and VBOs, in the style of Assignment 1.

Part A turns the “black & white” square into an interactive program with three windows. The squares rotate counter-clockwise in the main window, the ellipse sits in a subwindow with its own background and menu, and window 2 shows a breathing circle and a clockwise triangle whose colour changes with seven keys. One idle function drives all animation from real time; the main-window menu stops and starts it and recolours the white squares through a submenu; left clicks add breathing circles of random colours.

Part B shows the colour cube with depth testing. The user chooses SCALE, ROTATE or TRANSLATE, changes X, Y and Z with six keys, changes the delta with two keys and resets everything with one key. The matrix $M = T * R * S$ with $R = R_z * R_y * R_x$ is built on the CPU and applied in the vertex shader.

Table 5. Requirements and where they are met

Requirement	Where it is met
A1. Double buffering	GLUT_DOUBLE in main() (partA.cpp 406, 408); glutSwapBuffers() in every display function (190, 201, 214)
A2. Main window with the black & white square	createMainWindow() 329–358, buildMainScene() 148–157
A3. Ellipse in a subwindow with its own display and background	createSubWindow() 360–380 (glutCreateSubWindow 362), displaySub() 193–202
A4. Menu only in the subwindow for its background	subMenu() 260–272, attached in createSubWindow() 374–379
A5. Circle and triangle in “window 2”, colour keys	createWindow2() 382–399, keyboard2() 301–321
A6. Animation with an idle function	idle() 217–229, uniforms in vshaderA.glsl, setTransform() 141–146
A7. Main menu with Stop, Start and the Square Colors submenu	createMainWindow() 348–357, mainMenu() 237–243, squareColorMenu() 245–258
A8. Extra credit: circles at left clicks	mainMouse() 274–299, drawn in displayMain() 185–188
B1. Colour cube with corner colours	colorcube() 82–90, quad() 72–80 in partB.cpp
B2. Depth test	GLUT_DEPTH 289, 291; glEnable(GL_DEPTH_TEST) 234; GL_DEPTH_BUFFER_BIT 242
B3. Menu SCALE / ROTATE / TRANSLATE and $M = T * R * S$	menu 309–313, menu() 278–282, matrix in display() 244–247
B4–B6. Six axis keys, two delta keys, one reset key	keyboard() 255–276, resetTransformations() 164–172

Requirement	Where it is met
B7. Matrix built on the CPU, applied in the vertex shader	mat4 functions 92–162, glUniformMatrix4fv 249, vshaderB.glsl
B8. No camera or projection	vshaderB.glsl uses only <code>model * vPosition</code>
B9. Instructions are the only output	printInstructions() 184–203, called once in main() 315

Several directions follow naturally. The most direct one is a camera and a projection matrix in Part B, so that translation along *Z* changes the apparent size of the cube and the cube is no longer clipped at $z = \pm 1$. Rotations could be stored as quaternions to avoid the dependence on the order of the *X*, *Y* and *Z* angles. Part A could keep the state of each clicked circle (its own phase and colour) and let the user remove circles. Finally, lighting with normals would give the cube shading that depends on its orientation.

8. References

Journal articles (Literature Review). Authors, year, journal and DOI were checked in Crossref; the quartiles of the journals are given in Table 1.

Alexa, M. (2002) Linear combination of transformations. *ACM Transactions on Graphics*, 21(3), 380–387. <https://doi.org/10.1145/566654.566592>

Blythe, D. (2006) The Direct3D 10 system. *ACM Transactions on Graphics*, 25(3), 724–734. <https://doi.org/10.1145/1141911.1141947>

Cohen-Or, D., Chrysanthou, Y. L., Silva, C. T., and Durand, F. (2003) A survey of visibility for walkthrough applications. *IEEE Transactions on Visualization and Computer Graphics*, 9(3), 412–431. <https://doi.org/10.1109/TVCG.2003.1207447>

Fatahalian, K., and Houston, M. (2008) A closer look at GPUs. *Communications of the ACM*, 51(10), 50–57. <https://doi.org/10.1145/1400181.1400197>

Hartley, R. I. (1997) In defense of the eight-point algorithm. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(6), 580–593. <https://doi.org/10.1109/34.601246>

Jacobson, A., Baran, I., Popović, J., and Sorkine, O. (2011) Bounded biharmonic weights for real-time deformation. *ACM Transactions on Graphics*, 30(4), 1–8. <https://doi.org/10.1145/2010324.1964973>

Kavan, L., Collins, S., Žára, J., and O’Sullivan, C. (2008) Geometric skinning with approximate dual quaternion blending. *ACM Transactions on Graphics*, 27(4), 1–23. <https://doi.org/10.1145/1409625.1409627>

Kovar, L., Gleicher, M., and Pighin, F. (2002) Motion graphs. *ACM Transactions on Graphics*, 21(3), 473–482. <https://doi.org/10.1145/566654.566605>

Lloyd, D. B., Govindaraju, N. K., Quammen, C., Molnar, S. E., and Manocha, D. (2008) Logarithmic perspective shadow maps. *ACM Transactions on Graphics*, 27(4), 1–32. <https://doi.org/10.1145/1409625.1409628>

Myers, B. A. (1990) A new model for handling input. *ACM Transactions on Information Systems*, 8(3), 289–320. <https://doi.org/10.1145/98188.98204>

Myers, B. A. (1995) User interface software tools. *ACM Transactions on Computer-Human Interaction*, 2(1), 64–103. <https://doi.org/10.1145/200968.200971>

Myers, B., Hudson, S. E., and Pausch, R. (2000) Past, present, and future of user interface software tools. *ACM Transactions on Computer-Human Interaction*, 7(1), 3–28. <https://doi.org/10.1145/344949.344959>

Owens, J. D., Luebke, D., Govindaraju, N., Harris, M., Krüger, J., Lefohn, A. E., and Purcell, T. J. (2007) A survey of general-purpose computation on graphics hardware. *Computer Graphics Forum*, 26(1), 80–113. <https://doi.org/10.1111/j.1467-8659.2007.01012.x>

Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., and Phillips, J. C. (2008) GPU computing. *Proceedings of the IEEE*, 96(5), 879–899. <https://doi.org/10.1109/JPROC.2008.917757>

Slater, M., Drake, K., Davison, A., Kordakis, E., Billyard, A., and Miranda, E. (1992) A statistical comparison of two hidden surface techniques: the scan-line and Z-buffer algorithms. *Computer Graphics Forum*, 11(2), 131–138. <https://doi.org/10.1111/1467-8659.1120131>

Sumner, R. W., and Popović, J. (2004) Deformation transfer for triangle meshes. *ACM Transactions on Graphics*, 23(3), 399–405. <https://doi.org/10.1145/1015706.1015736>

Sutherland, I. E., Sproull, R. F., and Schumacker, R. A. (1974) A characterization of ten hidden-surface algorithms. *ACM Computing Surveys*, 6(1), 1–55. <https://doi.org/10.1145/356625.356626>

Course and technical sources.

Angel, E., and Shreiner, D. (2014) *Interactive Computer Graphics: A Top-Down Approach with WebGL*. 7th ed. Pearson.

Astana IT University (2026) *Computer Graphics Fundamentals. Assignment 2: Interaction, Windows, Menus, Animation and 3D Transformations*.

Batkuldinova, K. (2026) *Computer Graphics Fundamentals*. Lecture materials. Astana IT University, Moodle.

Kilgard, M. J. (1996) *The OpenGL Utility Toolkit (GLUT) Programming Interface, API Version 3*. Silicon Graphics.

Khronos Group (2009) *The OpenGL Graphics System: A Specification, Version 3.2 (Core Profile)*.

9. Appendix

9.1 Contents of the submission

File	Purpose
partA.cpp	Part A: windows, subwindow, menus, keyboard, mouse and animation
partB.cpp	Part B: colour cube, transformation menu, keys and the model matrix
vshaderA.glsl	Vertex shader of Part A (uniforms theta, scale, offset)
vshaderB.glsl	Vertex shader of Part B (uniform model)
fshader.glsl	Fragment shader, the same as in Assignment 1
InitShader.cpp, InitShader.h	Reads, compiles and links the shaders (from Assignment 1)
Makefile	Builds partA and partB
README.md	Short build and run instructions
report.md	This report
screenshots/	The figures of this report

9.2 How to build and run

```
make  
./partA  
./partB
```

Run the programs from this folder, because the shader files are loaded with relative paths.

9.3 Controls

Part A

Where	Input	Action
Main window	Right mouse button	Menu: Stop Animation, Start Animation, Square Colors → White / Red / Green
Main window	Left mouse button	Add a breathing circle of a random colour
Subwindow	Right mouse button	Menu: background Dark Blue / Dark Green / Gray / Light Yellow
Window 2	r, g, b, y, o, p, w	Circle and triangle red, green, blue, yellow, orange, purple, white
Main window, window 2	Esc or q	Quit

Part B

Input	Action
Right mouse button	Menu: SCALE, ROTATE, TRANSLATE
q / a	Increase / decrease X of the current transformation
w / s	Increase / decrease Y
e / d	Increase / decrease Z
+ / -	Double / halve the delta of the current transformation
r	Reset to the initial view (rotation -45° about X, 35° about Y) and the default deltas
Esc	Quit

9.4 The vertex shader of Part B

```
#version 150

in vec4 vPosition;
in vec4 vColor;
out vec4 color;

uniform mat4 model;

void main()
{
    color = vColor;
    gl_Position = model * vPosition;
}
```

9.5 Repository

The complete history of the work is available at <https://github.com/Altusha4/opengl-assignment-1>